Fixing Non-blocking Data Structures for Better Compatibility with Memory Reclamation Schemes

Md Amit Hasan Arovi
The Pennsylvania State University
University Park, USA
arovi@psu.edu

Ruslan Nikolaev
The Pennsylvania State University
University Park, USA
rnikola@psu.edu

Abstract

We present a new technique, Safe Concurrent Optimistic Traversals (SCOT), to address a well-known problem related to optimistic traversals with classical and more recent safe memory reclamation (SMR) schemes, such as Hazard Pointers (HP), Hazard Eras (HE), Interval-Based Reclamation (IBR), and Hyaline. Unlike Epoch-Based Reclamation (EBR), these (robust) schemes protect against stalled threads but lack support for well-known data structures with optimistic traversals, e.g., Harris' list and the Natarajan-Mittal tree. Such schemes are either incompatible with them or need changes with performance trade-offs (e.g., the Harris-Michael list).

SCOT keeps existing SMR schemes intact and retains performance benefits of original data structures. We implement and evaluate SCOT with Harris' list and the Natarajan-Mittal tree, but it is also applicable to other data structures. Furthermore, we provide a simple modification for wait-free traversals. We observe similar performance speedups (e.g., Harris vs. Harris-Michael lists) that were previously available only to EBR users. Our version of the tree also achieves very high throughput, comparable to that of EBR, which is often treated as a practical upper bound.

CCS Concepts: • **Theory of computation → Concurrent algorithms**.

Keywords: hazard pointers, non-blocking, wait-free, Harris' list, Natarajan-Mittal tree

ACM Reference Format:
Md Amit Hasan Arovi and Ruslan Nikolaev. 2026. Fixing Non-blocking Data Structures for Better Compatibility with Memory Reclamation Schemes. In *Proceedings of the 31st ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming (PPoPP '26), January 31 – February 4, 2026, Sydney, NSW, Australia.* ACM, New York, NY, USA, 15 pages. https://doi.org/10.1145/3774934.3786455

This work is licensed under a Creative Commons Attribution 4.0 International License.
PPoPP '26, Sydney, NSW, Australia
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2310-0/2026/01
https://doi.org/10.1145/3774934.3786455

1 Introduction

With the rise of multicore and manycore systems, parallelization becomes essential for improving performance across the entire software stack, from applications to operating system (OS) components. Non-blocking data structures, which are not dependent on mutual exclusion, are becoming increasingly popular due to their scalability, throughput, and latency characteristics. Unfortunately, many data structures that utilize non-blocking techniques conflict with fundamental assumptions of classical memory management. This is because memory cannot be promptly reclaimed after an object is removed from a data structure, as there may still exist stale pointers to the object due to ongoing operations. When using manual memory management, memory has to be safely reclaimed by freeing memory only when *all* ongoing operations with stale pointers complete. A number of *safe memory reclamation* (SMR) techniques were proposed for C, C++, and Rust. While we do not want to compromise data structure progress properties, it can be challenging to achieve several *highly desirable* properties simultaneously with non-blocking progress in SMR. A recent work [30] sheds more light onto this conundrum: it proves that it is *impossible* to achieve three highly desirable SMR properties simultaneously, i.e., only two properties can ever be achieved: **(A) robustness** (bounded memory usage), **(B) easy integration** (no dependency on any special APIs, such as roll-back mechanisms or OS-based primitives), and **(C) wide applicability** (compatibility with most data structures without changes).

While striving to have (A) for non-blocking progress,[1] many researchers also prioritize (C) because, intuitively, their goal is to support as many data structures as possible. However, we argue that sacrificing (C) rather than (B) by carefully adapting existing problematic data structures is a "necessary evil," which is a *crucial new insight*. First, (C) in its strongest form – applying to *all* data structures – is only currently known to exist in *blocking* SMR schemes [30]. Second, even if we accept (C) in its relaxed form – applying to *most* data structures – the absence of (B) would lead to either inability to achieve strict non-blocking progress in practice due to OS dependency [1, 2, 6, 8, 31], lack of universally-defined APIs [29], or both. Third, the lack of (C) implies that adaptations are required, not impossibility of the implementation.

[1] If memory usage is unbounded, no further system progress can be made once memory is exhausted, regardless of whether actual locks are present.

EBR [12] is a fast and easy-to-use approach but has a serious drawback that any stalled thread results in unbounded memory usage, i.e., it lacks (A). Once exhausting memory, the system must either reboot or block until memory is freed. HP [21] is an antipode of EBR: it is slower and more difficult to use but provides robustness with strict non-blocking guarantees. Prior to this work, well-known data structures such as Harris' linked list [14] with optimistic traversals and the Natarajan-Mittal binary search tree (BST) [24] were not known to work well with HP [3, 19],[2] which is due to HP's lack of the wide-applicability (C) property. A number of other recent schemes, including Interval Based Reclamation (IBR) [34], Hyaline-1S [26], and Hazard Eras (HE) [28], sometimes offer better performance but have similar issues to HP, though IBR and Hyaline-1S are generally easier to use in practice. HP++ [19] is another recent scheme. HP++ is slower than HP and, strictly speaking, makes compromises with both (B) and (C), but it still supports these two data structures. We are inspired by HP++'s success, which indicates that **many data structures can still be implemented**, despite earlier views about the limited applicability of SMR schemes in this category.

This paper introduces an alternative method, Safe Concurrent Optimistic Traversals (SCOT). We depart from the typical strategy of designing a "silver-bullet" SMR approach for the reasons described above. Instead, we argue that the problematic data structures can be modified to accommodate SMR schemes with strict non-blocking properties. SCOT's *key insight* is that optimistic traversals are still feasible with HP and other SMR schemes as long as at every traversal step, we perform a simple safety check, which allows us to proceed safely to the next data-structure object. We also discuss how to address our approach's limits for *wait-free* traversals.

We present SCOT for Harris' list and the Natarajan-Mittal tree and evaluate these data structures with HP, HE, IBR and Hyaline-1S. We also compare results to EBR and show that performance benefits can be preserved in the same manner.

2 Background

2.1 Non-Blocking Progress Guarantees

In the literature, several *non-blocking* progress properties are considered. With *obstruction-freedom*, progress is only guaranteed when a thread runs in isolation from others. *Lock-freedom* allows threads to interfere with each other while still guaranteeing that *at least* one thread makes progress in a finite number of steps. (It should be noted that good lock-free algorithms allow multiple threads to make concurrent progress in practice.) This should not be confused with "lockless" approaches that simply avoid *explicit* locks, where one preempted thread can still block all other threads. *Wait-freedom* is a much stronger property which requires that *all* threads make progress in a finite number of steps.

[2] A few issues were discussed in [3], but only the memory leak was addressed.

Most non-blocking data structures use *compare-and-swap* (CAS) for synchronization. CAS is an instruction which takes three arguments: a *pointer* to the shared variable to be updated, an *expected* value, and a *desired* new value; CAS atomically replaces the shared-variable value with the desired value if its current value equals the expected value.

2.2 Memory Reclamation

All SMR schemes used in this paper follow the same lifecycle: a removed node is first *retired*, and then reclaimed only once the scheme guarantees that no thread can access it anymore. Each SMR scheme maintains per-thread metadata (epochs, eras, or reserved pointers) to determine reclamation safety. Although SMR mechanisms differ, they implement the same two-step process: unlink and retire, then reclaim when safe.

2.2.1 Epoch-Based Reclamation (EBR). Epoch-Based Reclamation (EBR) [12, 15] ensures memory safety using a global epoch. Threads entering a critical section publish the current epoch, and retired nodes are stored in per-thread lists tagged with that epoch. The global epoch advances periodically, and nodes are reclaimed only after all active threads have moved beyond the retire epoch. The main limitation of EBR is its sensitivity to stalled or crashed threads, since the non-progressing thread can prevent epoch advancement and lead to unbounded memory accumulation.

2.2.2 Hazard Pointers (HP). Hazard Pointers (HP) [21] ensure memory safety by requiring each thread to publish any pointer it may dereference in a globally visible hazard slot. A retired object is reclaimed only after scanning all slots to verify that no thread still references it. This makes HP robust to stalled or preempted threads because it does not rely on global quiescence. The approach introduces overhead due to memory barriers and hazard-slot scans, and it also complicates traversal-heavy code. Despite these costs, HP remains widely used for strong safety guarantees.

In Section 3, we rely on the `protect` and `dup` operations (shown in Figure 1). `protect` safely retrieves an object by making a *reservation* in the global array; the idea is that other threads will not reclaim reserved pointers. Each local pointer has a unique *index*. Due to a small race window, HP verifies in a loop that the pointer has not changed after the reservation was made. `dup` copies an already protected pointer from one hazard slot to another, enabling safe transitions between traversal roles (e.g., from the *next* to the *current* pointer). This is crucial to avoid transient unprotected states. (An alternative to `dup` is index renaming via an indirection array, but from our observations, `dup` is generally cheaper.)

2.2.3 Hazard Eras (HE). Hazard Eras (HE) [28] optimize hazard pointers by using logical timestamps ("eras") instead of explicit pointer protection. Threads record the current era when accessing an object, and retired objects are tagged with a retire era; reclamation is safe once no thread holds an

```
1  node_t *protect(node_t* &var, int idx)
2    node_t *ret, *n = nullptr;
3    while ( (ret = var.load()) != n) do
4      hp[TID][idx] = ret;      // Also clear
5      n = ret;        // logical-deletion bits
6    return ret;

   // Duplicate HP_i to HP_j
7  node_t *dup(int i, int j)
8    node_t* p = hp[TID][i];
     // Release semantics
9    hp[TID][j] = p;
10   return p;
```

Figure 1. HP methods (TID is the current thread number).

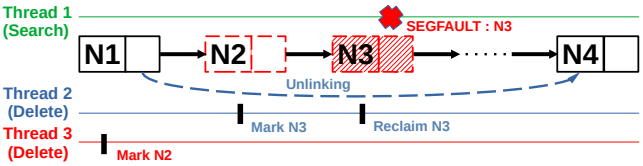


Figure 2. Unsafe optimistic traversal of Harris' linked list with HP: accessing nodes after N2 may cause SEGFAULTs.

access era that intersects with it. This temporal mechanism is an alternative to the direct pointer mechanism and reduces the number of memory barriers compared to HP.

Table 1. Subjective performance characterization of common non-blocking data structures and their compatibility with SMRs. (* HP column also applies to HE/IBR/Hyaline-1S.)

Data Structure	Fast	EBR	HP*	HP* with SCOT
Harris' Linked List [14]	✔	✔	✘	✔
Harris-Michael Linked List [20]	✓	✔	✔	✔
Fraser's Skip List [12]	✔	✔	✘	✔
Herlihy-Shavit Skip List [18]	✓	✔	✔	✔
Natarajan-Mittal Tree [24]	✔	✔	✘	✔
Ellen et al. Tree [11]	✘	✔	✔	✔

2.2.4 Interval-Based Reclamation (IBR). Interval-Based Reclamation (IBR) [34] extends era-based methods by tracking access *intervals* rather than individual accesses. Each thread maintains a birth and retire era, and a retired node is reclaimable once no thread's interval overlaps with its lifetime. IBR avoids explicit tracking of local pointers via indices, which simplifies its programming model.

2.2.5 Hyaline-1S. Hyaline-1S [26] is another SMR technique, which adopts the birth era approach from HE and IBR. Instead of retire eras, it maintains an object (reference) counter. Hyaline-1S reduces memory barriers by using reference counting only during reclamation. Each thread maintains a birth era, and an object is reclaimed only when its implicit lifetime interval does not overlap with any active thread's era, ensuring safety even with stalled threads. Reclamation is done by *any* thread, improving the performance.

2.3 Overview of Data Structures

Table 1 characterizes common data structures by speed and SMR compatibility, both original and with SCOT. We omit hash maps as they are simply arrays of Harris' or Harris-Michael lists. These data structures employ "logical" deletion, i.e., a node inside a data structure is first simply marked (by stealing one bit from a node pointer), and then unlinked from the data structure (i.e., "physically" deleted). Harris' list and Fraser's skip list can use read-only[3] optimistic traversals, which allow for safely continuing the iteration through the data structure, even when encountering logically deleted nodes. Moreover, a chain of *consecutive* logically deleted nodes is retired using one CAS operation. These optimizations improve performance and reduce contention.

However, as we discuss below, this approach is *incompatible* with HP. Harris-Michael lists and Herlihy-Shavit skip lists [18] use a different approach: they still use "logical" deletion. However, they mandate that logically deleted nodes are removed *immediately* after the first encounter and before proceeding further, including the search operation. Ellen et al. tree and the Natarajan-Mittal tree operate on tree branches rather than individual nodes. However, there is a similar distinction between them with respect to optimistic traversals and ability to remove multiple "logically" deleted branches.

As we show in the table, data structures with optimistic traversals are incompatible with HP, HE, IBR, and Hyaline-1S, a problem that we address in this paper.

2.4 Harris' and Harris-Michael Linked Lists

The main challenge in lock-free linked lists is allowing a deleting thread to mark a node as deleted and update the preceding node's next pointer. Timothy L. Harris solved this [14] by using helping threads during deletion. A deleting thread first marks a node as "logically" deleted by updating its next pointer, and then the node is "physically" removed by the deleting thread or a helper. (*Search* jumps over logically deleted nodes, so it is not a conflicting operation, but a concurrent *Insert* or *Delete* that requires the physical removal of the node is). Harris' list supports *optimistic traversals*, meaning that logically deleted nodes can still be traversed and bypassed by a search operation without immediate physical removal. This allows a thread to defer unlinking of logically deleted nodes until it reaches the intended destination node.

However, this approach poses challenges for HP (Figure 2). Suppose Thread 3 marks N2 for deletion but has not yet unlinked it; Thread 2 marks N3 and unlinks the entire chain between N1 and N4 (assuming all *consecutive* nodes are logically deleted). Meanwhile, Thread 1, traversing the list, reaches N2 before Thread 2 physically removes the chain and retires its nodes. Although Thread 2 retires N2, it remains in the HP *limbo list* (i.e., the list of retired but not-yet reclaimed nodes) since Thread 1 reserved it *before the physical deletion*.

[3]The original version [14] lacks the read-only optimization from [18].

```
1  struct {
2    node_t * Next; // Next node
3    key_t Key;  // Any key type
4  } node_t;
5  node_t * Head;    // List head
6  bool Insert(key_t key)
7    node_t *new =
       malloc(sizeof(node_t));
8    new->Key = key;
9    node_t **prev, *curr, *next;
10   while true do
11     if ( Do_Find(key, &prev,
         &curr, &next, false) )
12       free(new);
13       return false;
14     new->Next = curr;
15     if ( CAS(prev, curr, new) )
         return true;
16 bool Delete(key_t key)
17   node_t **prev, *curr, *next;
18   while true do
19     if ( !Do_Find(key, &prev,
         &curr, &next, false) )
20       return false;
21     if ( !CAS(&curr->Next,
         next, getMarked(next)) )
         continue;
22     if ( CAS(prev, curr, next) )
         smr_retire(curr);
23     return true;
24 void Init()
25   Head = malloc(sizeof(node_t));
26   Head->Next = nullptr;
27   Head->Key = ∞;
28 bool Search(key_t key)
29   node_t **prev, *curr, *next;
30   return Do_Find(key, &prev,
       &curr, &next, true);
31 bool Do_Find(key_t key, node_t
     ***p_prev, node_t **p_curr,
     node_t **p_next, bool srch)
32   node_t **prev, *curr, *next;
33   prev_next = nullptr;
34   prev = &Head;
35   curr = Head;
36   while curr != nullptr do
37     next = curr->Next;
38     if ( !isMarked(next) )
39       if ( curr->Key ≥ key ) break;
40       prev = &curr->Next;
41       prev_next = next;
42     curr = getUnmarked(next);
43   if ( prev_next && prev_next !=
       curr && !srch )  // Search(): skip
44     if ( !CAS(&prev, prev_next, curr
         ) goto 33 ;
       // Delete prev_next...curr chain
45   *p_curr = curr;
46   *p_prev = prev; *p_next = next;
47   return curr && (curr->Key==key);
```

Figure 3. Harris' list with optimistic traversals (w/o SMR).

Thread 2 retires N3, but since Thread 1 never reserves it, N3 is reclaimed causing Thread 1 to fail when accessing it.

Maged M. Michael modified Harris' approach [20] to address its incompatibility with HP: when a thread encounters a logically deleted node during a traversal, it *immediately* attempts to physically remove the node.

Why Michael's Approach Works. Accessing N2 is safe since its HP reservation precedes physical deletion. (To *successfully* make a reservation, HP verifies that the pointer from N1 to N2 remains intact.) After N2 is physically deleted, it is retired and placed into the HP limbo list. If we now attempt to reserve N3, HP erroneously succeeds as *the pointer from N2 to N3 stays intact, unlike the pointer from N1 to N2.* Thus, Michael's approach ensures that the successor of a marked node is never traversed: the marked node is unlinked from the list and the operation is restarted if the unlinking fails.[4]

While solving the HP problem, Michael's method increases the number of CAS operations, which increases thread contention and makes read-only wait-free traversals impossible.

In Figure 3, we present Harris' list without any SMR. The original Harris' list uses $-\infty$ and $+\infty$ sentinel nodes. We encode the first (pre-head) sentinel implicitly via `&Head` (L34); this removes one sentinel node without changing the semantics: Traversal still starts from `Head` and CAS acts on *link pointers* (`node_t**`). We keep a *pointer* to a link in `prev` so that CAS can update the predecessor field directly, including `&Head`, making the first sentinel unnecessary. Using `prev = &curr->next` allows the algorithm to perform `CAS(prev, curr, new)` uniformly, whether `prev` refers to `Head` or to an internal node. We use `node_t***` only to return the predecessor's link address (`node_t**`) to the caller, since `CAS` operates on the address of the variable being updated, exactly as in the original Harris algorithm.

The `Insert` method inserts a new node into the list. We slightly diverge from the original version [14] by using an optimization [18] which enables read-only search operations. Before inserting a new node, `Insert` checks if the key of the to-be-inserted node exists via `Do_Find`. The new node gets inserted unless its key is already present in the list. `Delete` removes the node from the list. If the key is found in the list, the node is logically deleted at L21 by marking `next`, and then one attempt is made to unlink it from the list at L22.

In `Search`, the underlying `Do_Find` operation skips L43-L44. In `Insert` and `Delete`, `Do_Find` locates the right position for the key. If a chain of marked nodes is found, the algorithm attempts to clean it up using CAS at L44. **If we integrate HP without any changes, L37 may crash.**

3 Design

3.1 Bird's-Eye View

The crux of the problem is HP's inability to properly track physically deleted nodes in Harris' list (or tagged edges in the Natarajan-Mittal tree) while traversing logically deleted nodes. However, if there were some way to confirm that the following logically deleted node has not yet been physically deleted, optimistic traversals would still be feasible.

In Figure 2, N2 is still *safe* to access because it is not physically deleted, i.e., N1 still points to N2. Moreover, according to Figure 3, we always make sure that the node to the left is not logically deleted and thus *never unlink nodes in the middle of the chain*, i.e., we can delete just N2, or both of N2 and N3, but we never delete N3 while keeping N2 in the list. In other words, we remove a prefix of the chain.

Combining these observations, we gain a crucial insight that helps solve the issue with HP: It is still safe to access N3 and the following nodes as long as at every step, i.e., after making an HP reservation for the next logically deleted node, we verify that N1 still points to N2. In other words, once we reach N2, we declare it to be a "dangerous zone." Until we reach the end of the chain of consecutive logically deleted nodes, we are going to stay in the dangerous zone and perform additional checks. More specifically, after retrieving N3's pointer and creating its HP reservation, we check that N1 still points to N2. We will continue to perform these checks at every iteration until we reach the end of the chain.

[4] Another way to think about it is that if we remove nodes one by one, N3 takes place of N2, i.e., right after N1. N3 will be the first logically deleted node in the chain when making HP reservation, which is still safe to access.

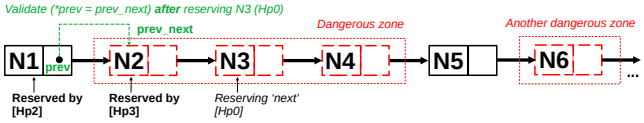


Figure 4. SCOT for Harris' list: validating *prev = prev_next at every iteration while traversing via the dangerous zone.

If the check fails, we cannot proceed further and need to restart the search operation from the very beginning.

SCOT makes several **key assumptions**: (1) The data structure supports a *deferred* deletion of "logically" deleted nodes and (2) For a logically deleted node to be physically unlinked, its predecessor on the traversal path must not itself be logically deleted. While uncommon in trees or lists, unlinking a node with multiple predecessors requires extra care if some predecessors are valid while others are logically deleted.

3.2 SCOT for Harris' List

In Figure 4, we present our approach for Harris' list. Compared to Michael's approach, we need an additional hazard pointer index (Hp3) which protects the first *unsafe* node, i.e., logically deleted node. This extra hazard pointer prevents the ABA problem, in case the unsafe node gets recycled while we are traversing the list. By holding this extra hazard pointer, we can solely rely on pointer comparison. In other words, Hp3 protects N2 from our example above irrespective of where we currently are in the chain of logically deleted nodes. Like in Michael's approach, we need Hp2, which protects the `prev` area. We note that in Harris' list, it will be the last *safe*, i.e., not logically deleted node.

We present the pseudocode of SCOT-augmented `Do_Find` in Figure 5. We show a simple version (on the left) and a version that unrolls the loop for better performance (on the right). We use `protect` and `dup`, discussed in Section 2.2.2. It is crucial to use `dup` such that the old HP index has a lower numerical value than the new HP index (e.g., Hp0 to Hp1). This allows to avoid a small race window when iterating in the same (ascending) order of indices in HP's `retire`. Specifically, when advancing from `next` to `curr`, we copy Hp0 to Hp1, but we cannot do it the opposite way. This ordering is critical because duplicating in the reverse direction (Hp1 → Hp0) would create a window where Hp0 temporarily points to stale data before being safely updated, potentially leading to unsafe access if a concurrent reclamation occurs. By maintaining the ascending order (0→1, 1→2, etc.), we ensure that each pointer remains protected throughout the traversal.

Our version on the *left* introduces simple changes to Harris' original list. Specifically, L15 makes sure that the last safe node still points to the first unsafe node while traversing.

The tricky part is avoiding L13-L14 (copying to Hp3) for *unmarked* nodes, which would add a memory barrier absent in the Harris-Michael approach, potentially increasing the algorithm's cost. We resolve this (on the *right*) by unrolling and splitting the loop into two phases: (1) *iterating through the safe zone* and (2) *iterating through the dangerous zone*. Phase 1 only duplicates (shifts) `prev` and `curr` hazard pointers from `curr` and `next`, respectively. This is similar to the Harris-Michael algorithm. Upon leaving Phase 1, `curr` is duplicated (Hp3) in L49. In Phase 2, we no longer duplicate `curr` into `prev` (Hp2), which gives us benefit over the Harris-Michael approach in the dangerous zone.

In the dangerous zone of logically deleted nodes, Figure 5 runs a check in L55 to ensure that the reservation made in L54 is valid. Once the chain (or its subchain) gets unlinked, L55 fails, and we restart the operation from the very beginning. We use green to highlight all major changes related to SCOT and pink to highlight the dangerous zone traversal.

3.2.1 Recovery Optimization.

For simplicity, we have stated that when validation fails, we restart from the beginning of the list. While this is acceptable, this validation check is not always critical: the last safe node may simply point to another node now (e.g., when a new node is inserted or the chain of logically deleted nodes is already eliminated by a concurrent thread). In such cases, we simply escape from the dangerous zone and continue to the new node. We must, however, ensure that the last safe node is still not logically deleted. When it is deleted, the last safe node (`prev`) is in the dangerous zone itself (though still safe to access due to a prior reservation), so we go back to the beginning of the list. In Section 5, Harris' linked list implements this optimization.

3.2.2 Why Not Change SMR?

In Figure 5, we introduced the SCOT logic directly in Harris' list rather than SMR. While it is feasible to extend the HP API, there are major caveats. For example, Figure 5's optimized version calls 2x `dup` outside the dangerous zone, and 1x `dup` in the dangerous zone (not counting `protect` calls in both cases). If we attempt changing SMR, we will end up with the basic version which needs 3x `dup` everywhere, not counting `protect`. An extra `dup` induces a cost, not present with Michael's approach, which needs 2x `dup`. Moreover, empirically, we found that the recovery optimization (Section 3.2.1) is beneficial for Harris' list. Conversely, in the Natarajan-Mittal tree, the recovery optimization does not help improve performance for various key ranges, primarily because of the hierarchical structure of the tree, i.e., the tree is likely diverging substantially anyway.

Taking all these factors together, it is better to **adapt a given data structure** based on its internal characteristics.

3.3 SCOT for the Natarajan-Mittal Tree

In the Natarajan-Mittal BST [24], all *actual* keys and values are located in the leaf nodes, while the internal nodes contain only *routing* keys used to direct the traversal through the tree. The Natarajan-Mittal BST incorporates a concept similar to Harris' logical deletion through the use of *tagging* (applied to edges or sibling nodes) and *flagging* (applied to leaf nodes).

```
1  bool Do_Find(key_t key, node_t
     ***p_prev, node_t **p_curr,
     node_t **p_next, bool srch)
2    node_t **prev, *curr, *next;
3    prev_next = nullptr;
4    prev = &Head;
5    curr = hp.protect(Head, Hp1);
6    while curr != nullptr do
7      next = hp.protect(curr->Next,
         Hp0);
8      if ( !isMarked(next) )
9        if ( curr->Key ≥ key )
           break;
10       prev = &curr->Next;
11       prev_next = next;
         // Copy Hp1[curr] to Hp2
12       hp.dup(Hp1, Hp2);
13       // Copy Hp0[next] to Hp3
14       hp.dup(Hp0, Hp3);
       // Dangerous zone: check
       // the last safe node (N1) still
       // points to 1st unsafe (N2)
15     else if ( prev->load() !=
         prev_next ) goto 3;
16     curr = getUnmarked(next);
       // Copy Hp0[next] to Hp1
17     hp.dup(Hp0, Hp1);
     // Skip for Search()
18   if ( !srch && prev_next &&
       prev_next != curr )
19     if ( !CAS(&prev, prev_next,
         curr ) goto 3 ;
20     Do_Retire(prev_next, curr);
21   *p_curr = curr;
22   *p_prev = prev; *p_next = next;
23   return curr && (curr->Key ==
       key);
24 void Do_Retire(node_t *from,
     node_t *to)
25   do  // Retire the deleted chain
26     node_t *n = getUnmarked(
         from->Next);
27     smr_retire(from);
28     from = n;
29   while (from != to);
     // Hp0 = 0: Next node (next)
     // Hp1 = 1: Current node (curr)
     // Hp2 = 2: Last safe node (prev)
30   // Hp3 = 3: First unsafe node
```

```
31 bool Do_Find(key_t key, node_t
     ***p_prev, node_t **p_curr,
     node_t **p_next, bool srch)
32   node_t **prev, *curr, *next;
33   prev_next = nullptr;
34   prev = &Head;
35   curr = hp.protect(Head, Hp1);
36   next = hp.protect(curr->Next,
       Hp0);
37   while true do
38     do
39       if ( curr && curr->Key ≥
           key ) goto 57;
40       prev_next = nullptr;
41       prev = &curr->Next;
         // Copy Hp1[curr] to Hp2
42       hp.dup(Hp1, Hp2);
43       curr = getUnmarked(next);
44       if ( !curr ) goto 60;
         // Copy Hp0[next] to Hp1
45       hp.dup(Hp0, Hp1);
46       next = hp.protect(
           curr->Next, Hp0);
47     while !isMarked(next);
48     // Copy Hp1[curr] to Hp3
49     prev_next =
         hp.dup(Hp1,Hp3);
       // Dangerous zone: check
       // the last safe node (N1) still
       // points to 1st unsafe (N2)
50     do
51       curr = getUnmarked(next);
52       if ( !curr ) goto 57;
         // Copy Hp0[next] to Hp1
53       hp.dup(Hp0, Hp1);
54       next = hp.protect(
           curr->Next, Hp0);
55       if ( prev->load() !=
           prev_next ) goto 33;
56     while isMarked(next);
     // Skip for Search()
57   if ( !srch && prev_next &&
       prev_next != curr )
58     if ( !CAS(&prev, prev_next,
         curr ) goto 33;
59     Do_Retire(prev_next, curr);
60   *p_curr = curr;
61   *p_prev = prev; *p_next = next;
62   return curr && (curr->Key ==
       key);
```

Figure 5. SCOT for Harris' list with HP: showing Do_Find only (**left** side: unoptimized, **right:** with the unrolled loop).

The Natarajan-Mittal BST makes one crucial insight: a chain of tagged edges can be eliminated with a single CAS operation. Moreover, for the search operation, the chain of tagged edges can be just skipped over. This also makes this tree somewhat faster than Ellen et al. tree, depending on the exact workload; both trees were previously evaluated in [19], and the Natarajan-Mittal tree is almost always faster.

In essence, the Natarajan-Mittal BST makes an optimization similar to Harris' linked list – an optimistic traversal of the chain of tagged edges. While optimistic traversals are beneficial in terms of performance, they are also *fundamentally* incompatible with HP and many other robust schemes for the same reason as Harris' list is incompatible with HP.

The Natarajan-Mittal BST implements `Insert`, `Delete`, and `Search` operations, which are similar to those of Harris' list, except that the tree traversal is typically faster than a linear search in the list. `Search` starts at the root and traverses through internal nodes (by comparing keys) and continues until reaching the leaf node with the actual key. If the key is not found in the leaf node, the operation fails. `Insert` traverses the tree by using a similar, internal `Do_Find` method. If the key is found, `Insert` fails. Otherwise, `Insert` allocates a new internal node, which has *left* and *right* pointers initialized to the leaf node that was found as well as to the leaf node that is about to be inserted. Then the pointer to the old leaf node is being replaced by CAS to the new internal node. `Delete` also traverses the tree by using `Do_Find`. If the key is not found, it fails. Otherwise, `Delete` logically deletes the leaf node by changing the corresponding edge's "flag" bit. Right after this, the sibling of the flagged leaf is "tagged". Finally, an internal `CleanUp` procedure must be called to prune tagged edge(s), as shown in Figure 6. The successor (internal) node will be then replaced with the sibling node.

The SCOT solution is largely similar to that of Harris' list. Per original terminology [24], the *successor* node is the node with last untagged edge which comes from *ancestor*. Those can be any distance apart from the *parent* node, which is an immediate parent of the *leaf* node. Tagged edges constitute the "dangerous zone" (Figure 6), already discussed in the context of Harris' list; they will reside between the successor and the parent. *At each step, we verify that the ancestor still points to the successor as we traverse via the dangerous zone.*

In the SCOT version of the Natarajan-Mittal BST, five hazard pointers are used, each mapping directly to a specific role in the traversal. **Hp0** protects the current child pointer being followed. **Hp1** protects the current leaf candidate. **Hp2** protects the parent of the leaf. **Hp3** protects the successor node, which is the first node reachable through an untagged edge and marks the entrance to the tagged (dangerous) zone. **Hp4** protects the corresponding ancestor node whose child pointer should continue to reference the successor.

3.4 Wait-Free Traversals

The remaining piece is related to EBR's original wait-free traversal guarantees. With our approach, optimistic traversals may need to occasionally restart the search operation from the beginning if, due to overlapping modifications, the state of the data structure diverges significantly and cannot be recovered locally. This guarantees lock-free but not wait-free progress. This limitation is very similar to that of HP++'s existing solution which also does not support wait-free optimistic traversals due to potential restarts.

However, it is not hard to design an approach which brings wait-free traversals back to HP and other robust schemes

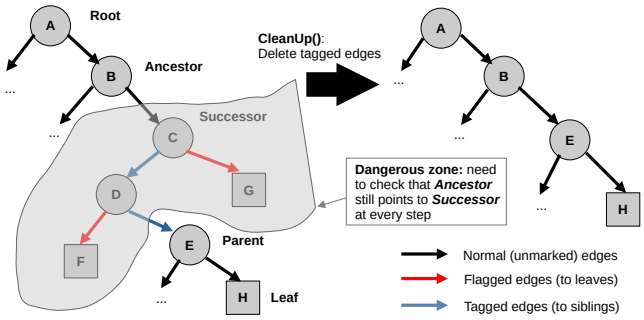


Figure 6. Natarajan-Mittal Tree: Tagged and Flagged Edges.

without resorting to a full-blown wait-free data structure. (We evaluate Harris' list in Section 5 using this approach.)

Our idea is based on a custom fast-path-slow-path method for `Search`. On every iteration, `Insert` and `Delete` check if any searching thread needs help (by running `Help_Threads`). If `Help_Threads` returns true, `Insert` or `Delete` will proceed with the `Slow_Search` slow-path procedure (see below), which is executed prior to the actual insertion or removal.

Figure 7 shows pseudocode. The original `Do_Find` method is still used as is for `Insert` and `Delete` because wait-free guarantees are only provided for traversals. The `Search` operation will run the original `Do_Find` for a few iterations (fast path), after which it will call `Request_Help` to indicate that the thread needs assistance by other threads, via a per-thread record entry (`thrdrec_t`). `Request_Help` initializes the search key (L28) and then updates `helpTag` (L30), so that other threads are aware that the thread expects help.

On input, `helpTag` contains the current slow-path cycle number; this is needed to prevent any belated updates by helpers intended for past cycles. On output, the same variable, `helpTag`, will contain the result of the search operation. To differentiate inputs and outputs, we reserve one bit (*IsInput*), whereas *Value* represents the actual input tag or output.

The output does not keep track of tags because it is only used by the helpee which is aware of its slow-path cycle. Our approach is specifically tailored for traversals; it avoids non-standard instructions (only a regular CAS) and complexity related to dynamically-allocated descriptors [32, 33].

Both helpers and the helpee run the same `Slow_Search` operation. On every iteration, it checks (L34) whether *any* thread has already produced an output (L37), or whether the slow-path input tag has changed since the helper began (L36). `Slow_Search`'s return value is only used by the helpee itself. Since the input tag can only be changed by the *helpee*, the return value in L36 is irrelevant: The L36 condition applies only to *helpers*, who ignore `Slow_Search`'s return value.

4 Correctness

We analyze the SCOT technique to establish two core properties: (1) **Safety:** no thread accesses previously reclaimed

```
1  struct thrdrec_t {
   // === Private Fields ===
2    int nextCheck;          // = DELAY
3    int nextTid;            // Thread ID
4    uint localTag;   // Slow-Path Tag
5    // === Shared Fields ===
6    key_t helpKey;      // Key (Input)
7    // One bit (.IsInput) is reserved to
8    // differentiate Input/Output
9    uint helpTag;     // Tag (Input) or
10 };                  // Result (Output)
11 thrdrec_t WF[MAX_THREADS];
12 bool Help_Threads(key_t *p_key,
     uint *p_tag, int *p_tid)
13   if ( --WF[TID].nextCheck )
14     return False;
15   WF[TID].nextCheck = DELAY;
16   currTid = WF[TID].nextTid;
17   WF[TID].nextTid = (currTid + 1)
       % MAX_THREADS;
18   if ( currTid == TID )
19     return False;
20   tag=WF[currTid].helpTag.load();
21   if ( !tag.IsInput ) return False;
22   key=WF[currTid].helpKey.load();
23   if ( WF[currTid].helpTag.load()
       != tag ) return False;
24   *p_key = key; *p_tid = currTid;
25   *p_tag = tag;
26   return True;
27 uint Request_Help(key_t key)
28   WF[TID].helpKey.store(key);
29   tag = WF[TID].localTag;
30   WF[TID].helpTag.store({
       .Value = tag, .IsInput = 1});
31   WF[TID].localTag = tag + 1;
32   return { tag, 1 };      // Input
     // Note: For the current thread
     // itself (helpee), helpTid = TID
33 bool Slow_Search(key_t key,
     uint tag, int helpTid)
       // Like Fig. 5's Do_Find, but
       // every iteration (L39, L51)
       // checks if result is available:
34     r=WF[helpTid].helpTag.load();
35     if ( r != tag )
         // Return value is irrelevant
         // for a different input tag
36       if ( r.IsInput ) return False;
         // DONE: output value is
37       return r.Value;  // available
38     goto Do_Find:33;  // Figure 5
39     ...
       // If the key has been found
40     ret = True/False;
       // This thread has found the
       // result itself => notify others
41     WF[helpTid].helpTag.CAS(tag,
         { .Value = ret, .IsInput = 0});
42     return ret;
```

Figure 7. Wait-Free Traversals with SCOT.

memory, and (2) **Lock-Freedom:** the system guarantees global progress. We also analyze SCOT's wait-free traversal extension (Figure 7). We prove that the Search(key) operation terminates in a finite number of steps for all threads (wait-freedom) while preserving safety and linearizability.

Without loss of generality, for simplicity, we assume a sequentially consistent memory model in our arguments. Also, Theorems 2 and 3 **consider only Harris' list**, but similar arguments apply to the Natarajan-Mittal tree as well.

4.1 Memory Bounds

Theorem 1 (Bounded Memory Overhead)**.** *For a SCOT-based implementation of Harris' list and the Natarajan-Mittal tree, the total memory usage with HP is $O(|D| + N)$, where $|D|$ is the number of live nodes and N is the number of threads.*

Proof. Each thread uses a fixed number H of hazard-pointer slots (H=4 for Harris' list and H=5 for the Natarajan-Mittal tree), so *at most* $H \cdot N$ nodes are reserved at any time. Let R be a constant defining a limbo-list scanning threshold (e.g., 128). Removed nodes are retired and placed into a per-thread limbo list, and reclamation is triggered only after a thread has retired R nodes. Between reclamation cycles, each thread can accumulate at most R unreclaimed nodes, giving at most $N \cdot R$ additional retired nodes system-wide.

During limbo-list scanning, any retired node not protected by a hazard pointer is freed; at most $H \cdot N$ previously retired nodes remain unreclaimed, and $N \cdot R$ additional nodes

can still be in the limbo list before scanning completes. The total number of unreclaimed nodes at any point of time is $HN + NR = O(N)$. Including $|D|$ reachable nodes, the total memory bound becomes $O(|D| + N)$. □

4.2 Safe Optimistic Traversal

Theorem 2 (HP Compatibility & Traversal Safety). *When a thread T traverses a dangerous zone consisting of the marked sequence $\langle N_k, N_{k+1}, \ldots, N_t \rangle$, we assign hazard pointers so that* `HP(T,2)` *protects the last safe predecessor N_{k-1},* `HP(T,3)` *protects the head of the marked region (the first unsafe node N_k), and* `HP(T,1)` *protects the current node N_i during traversal. Under this assignment, T never dereferences a reclaimed node.*

Proof. When T detects the marked region, it protects the last unmarked predecessor N_{k-1} and the first marked node N_k, thus making sure that both of them will not be reclaimed. As T traverses the marked chain, each node N_i and its successor are hazard-protected, and T consistently validates that the predecessor link in N_{k-1} still points to N_k. The risk of unsafe memory accesses exist *only* after physical unlinking. Any physical unlinking requires the predecessor node to remain unmarked (Figure 3, L22). Thus, only N_{k-1}'s link can change, irrespective of the actual removal operation among nodes in the marked sequence. Due to being reserved, N_k cannot get recycled, and when N_{k-1}'s link changes, it points to some other node, causing validation to fail. Traversal restarts (Figure 5, L15 and L55) before reaching anything that has already been reclaimed in the marked sequence. □

4.3 Lock-Freedom

Theorem 3 (Lock-Free Updates). *In the SCOT-based design, all update operations (Insert and Remove) are lock-free.*

Proof. Each update proceeds through repeated attempts (Figure 3, L11 and L19). Assuming a finite list length, in each attempt, SCOT runs Do_Find, which consists of a bounded number of local steps to reach the target location (reads and validation). A restart (Figure 5, L15 and L55) occurs only when detecting a changed link. The link changes *only* during another successful update: Insert (Figure 3, L15), Delete (Figure 3, L21), or Do_Find (Figure 5, L19 and L58).

There is one attempt in Insert (L15) or Delete (L21) to change the link after Do_Find; a failure means that another thread has changed the same location first, indicating global progress. If the current thread indefinitely fails, every failure is caused by a successful update by another thread. While any thread can starve indefinitely in Insert, Delete, or Do_Find, there is always at least one thread that makes progress. □

4.4 Wait-Free Traversals

Lemma 4 (Finite Wait Time). *Each thread that called Request_Help is helped by at least one thread after at most $N \cdot DELAY$ update operations, where N is the number of threads.*

Proof. Insert and Delete call Help_Threads every iteration. They assist exactly *one* thread after DELAY calls of Help_Threads (`nextCheck` in Figure 7, L13 amortizes checks). Threads are being served in a circular fashion (L17). Since the number of threads is N, the thread under consideration will be served after N calls amortized by the *DELAY* factor. □

Lemma 5 (Uniqueness). *For any slow-path tag t, only one thread writes an output; no late helper replaces a newer value.*

Proof. A request is posted by Request_Help, which writes $helpTag \leftarrow \langle v, \text{In} \rangle$ for a per-thread version v and returns $\langle v, \text{In} \rangle$ to the requester. A helper publishes a result only via the atomic transition at L41: $\text{CAS}\big(helpTag, \langle v, \text{In} \rangle, \langle r, \text{Out} \rangle\big)$. Since CAS compares against the exact input tag $\langle v, \text{In} \rangle$, at most one helper succeeds for the same version v. After that, *helpTag* changes to $\langle r, \text{Out} \rangle$, so any remaining helper observing $\langle ..., \text{Out} \rangle$ will fail the CAS. Future requests use strictly increasing version numbers ($v' > v$), ensuring that stale helpers fail the CAS on *helpTag*, which is now $\langle v', \text{In} \rangle$. □

Lemma 6 (Bounded Slow Path). *Slow_Search terminates after at most $O(N^2 \cdot DELAY + depth(L))$ operations, where $depth(L)$ is the maximum depth of the list.*

Proof. Slow_Search may need to restart due to a structural change, which implies that another thread completes an *update* operation. On each iteration, Slow_Search() re-reads the requester's *helpTag* (L34) to detect completion by another thread, i.e., when another thread completes CAS in L41, at which point the loop terminates immediately.

Per Lemma 4, at least one thread assists the helpee after $N \cdot DELAY$ update operations. This thread may also get stuck in its Slow_Search. Consequently, another thread after $N \cdot DELAY$ operations will join. After $O(N^2 \cdot DELAY)$ operations, *all* threads will be in Slow_Search, which will be then unobstructed and conclude after $depth(L)$) steps. □

Theorem 7 (Wait-Free Search). *Search(key) is wait-free.*

Proof. Search executes the fast path for a finite number of restarts M, which bounds its execution time to $O(M \cdot depth(L))$. Subsequently, it calls Request_Help and then switches to Slow_Search, which is bounded per Lemma 6. □

Theorem 8 (Linearizability). *Search(key) is linearizable.*

Proof. Both the fast and slow paths preserve the same validation and update semantics as the underlying lock-free data structure. A successful search linearizes at the read that observes a matching key under a valid link predicate, and an unsuccessful search linearizes at the final validated predecessor–successor relation. When help is provided, the helper executes exactly the same steps in the same order. □

5 Evaluation

Our benchmark reuses and substantially extends the test harness from [28], which already implements the Harris-Michael list (HMList) and a few SMR schemes: EBR, HP, and HE. We implemented Harris' list (HList) and the Natarajan-Mittal tree (NMTree) from scratch by carefully considering all known optimizations. We also added IBR [34] and Hyaline-1S [26] SMR schemes into the benchmark. We evaluated the original HP version [21] without extra optimizations. We also implemented and evaluated an optimized HP version (HPopt), which captures a local snapshot of shared data prior to the limbo list scanning [26]. HP vs. HPopt shows a substantial difference in some tests. Finally, we implemented a similar optimization for HE and IBR (not applicable to Hyaline-1S), as these schemes also benefit from it.

We did not implement HP++ [19] due to its substantial API differences (i.e., lack of an easy integration). HP++ is already known to perform mostly worse than HP for the *same* data structure [19], i.e., HP++ shows some performance benefits only when comparing different data structures (the Harris-Michael HP list vs. Harris' HP++ list). However, this comparison is largely irrelevant as we are now able to implement Harris' list and other data structures in HP directly.

We evaluate HMList, HList, and NMTree under different reclamation strategies: NR (no reclamation, leak memory), EBR, HP, HPopt, IBR, HE, and HLN (Hyaline-1S). For HList and NMTree, we use SCOT when evaluating them for all SMR schemes except NR/EBR. For HList, we present a version with wait-free traversals (Section 3.4); the lock-free version yields almost identical results. These algorithms were previously infeasible for HP/HPopt/IBR/HE/Hyaline-1S [3, 19].

We calibrated all SMR schemes to maximize throughput while minimizing not-yet-reclaimed objects. We found that amortizing limbo list scans at a frequency of one scan per 128 retire calls works well for EBR, HP, HPopt, HE, IBR, and Hyaline-1S. Also, for EBR, IBR, HE, and Hyaline-1S, we set the epoch increment frequency to 12 times the thread count.

We run experiments on Ubuntu 22.04.5 LTS using an AMD EPYC 9754 (Zen 4) system with 128 physical cores (256 hardware threads w/ hyperthreading enabled), 384 GiB of RAM, and a maximum clock speed of 3.10 GHz. The benchmark is written in C++ and compiled using Clang++ 14.0.0 with -O3 optimizations because its compiled code tends to perform slightly better than the code generated by Ubuntu's (default) GCC 11.4.0. For memory allocation, we use **Microsoft's `mimalloc`** [22] since it scales much better in multi-threaded code compared to glibc's stock malloc. (To stress test algorithms, we also ran all tests with glibc's stock malloc.)

We note that recent AMD EPYC architectures, including the evaluation server, show significantly improved HP performance due to reduced memory barrier overheads, the main prior bottleneck. AMD Zen 3 already introduced key cache-system optimizations that reduced these overheads, and Zen 4 further advances the architecture with improved cache-line sharing and enhanced multithreaded performance [23]. **Thus, our HP results, especially HPopt, are better than usual and often close to EBR.**

We measure both throughput and memory overhead. Memory overhead represents the number of not-yet-reclaimed objects, which are nodes that remain in an unreclaimed state due to the use of SMR. To calculate memory waste, we periodically record the average number of not-yet-reclaimed nodes in every thread. *We skip memory overheads for Hyaline-1S due to its global nature of reclamation, making it hard to calculate the number of unreclaimed objects locally per thread using this low-overhead mechanism which works for other SMR schemes.*

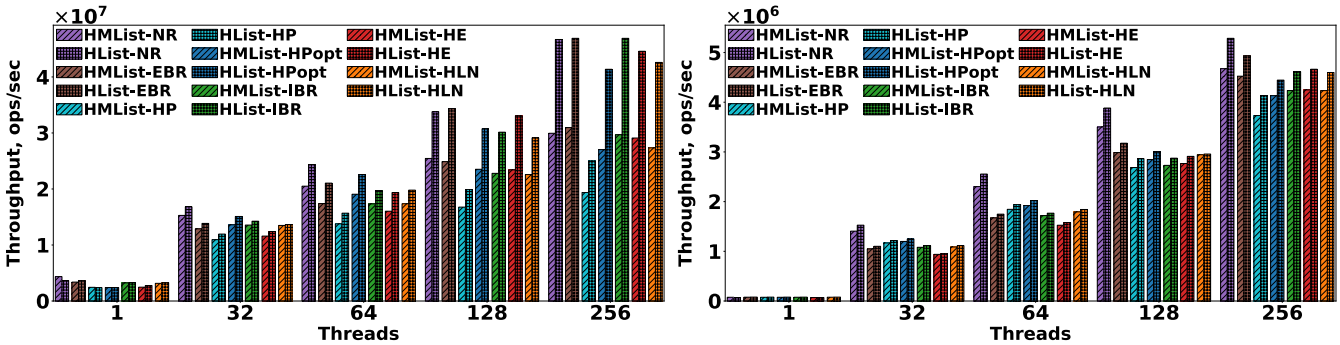
Each benchmark begins with prefilling the data structure with unique keys using 50% of the specified key range. We conduct 5 independent runs, each lasting 10 seconds. The median of all runs is used as the final measure for both throughput and memory overhead. (Across all experiments, the relative standard deviation remains mostly below 1%.) Our experiments use 1, 16, 32, 64, 128, 256, and 384 threads to evaluate performance for different levels of concurrency (384 threads represent oversubscription). Additionally, we test varying key ranges based on the data structure. For list, we use key ranges of 512 and 10,000. For tree, we use key ranges of 128, 100,000, and 50,000,000. We present results for a common workload, 50% read - 50% write (mixed reads and writes). We also measured 90% read - 10% write (read-dominated) and 50% insert - 50% delete (write-only) workloads, but we omit them as they exhibit largely similar trends.

All throughput figures include a special NR (*no reclamation*) baseline which simply leaks memory and demonstrates the "upper bound" for performance. In some cases, EBR and other SMRs can outperform this baseline, e.g., when the cost of reclaiming memory is much lower than that of performing fresh allocations. *We also noticed that NR may outperform SMRs, including EBR, under oversubscribed scenarios.*

In Figure 8, we present throughput results for the Harris-Michael list and Harris' list under different SMR schemes in a workload consisting of 50% read and 50% write operations. We observe that Harris' list consistently outperforms the Harris-Michael list for lower key ranges (512). As the key range increases to 10,000, the performance gap narrows, but Harris' list still maintains its throughput advantage. IBR, Hyaline-1S, HE, and HPopt show excellent performance on Harris' list with the SCOT traversal, often approaching that of EBR. The original HP scheme demonstrates lower throughput than its optimized counterpart (HPopt) in all tests.

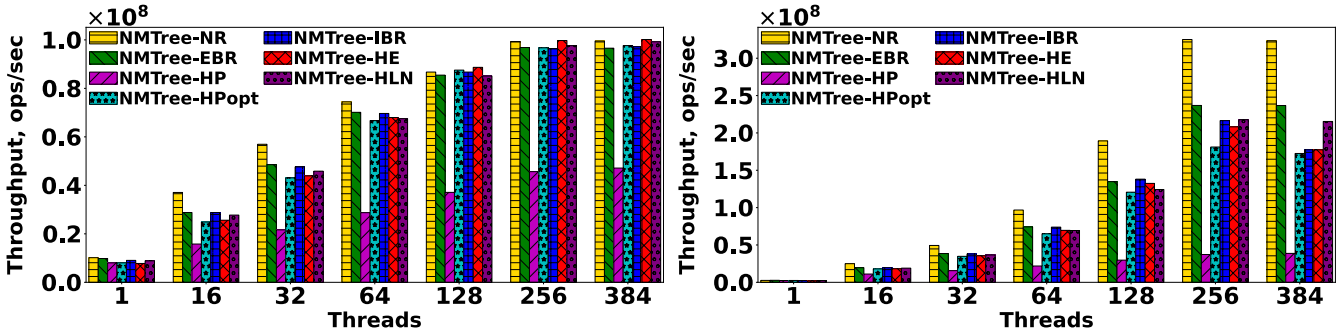
In Figure 10, we measure the memory overhead by showing the number of retired but not-yet-reclaimed nodes for Harris-Michael and Harris' lists. As expected, both HP and HPopt consistently maintain the smallest number of such unreclaimed objects compared to all other SMR schemes.

Figure 9 presents the throughput of the Natarajan-Mittal tree under a 50% read and 50% write workload. For the smaller



(a) Key Range = 512 **(b)** Key Range = 10,000

Figure 8. Linked List Throughput (50% Read - 50% Write): *higher is better.* (Skipped 384 threads; close to 256.)



(a) Key Range = 128 **(b)** Key Range = 100,000

Figure 9. NMTree Throughput (50% Read - 50% Write): *higher is better.*

key range (128), the baseline performance (without reclamation) is comparable to that of the various reclamation schemes, with only a small performance gap observed. In contrast, Figure 9b highlights the scalability of the tree at higher key ranges, where the Natarajan-Mittal tree achieves up to **240 million operations per second** at 256 threads using the EBR scheme. Hyaline-1S also performs competitively, reaching **210 million operations per second**, making it the closest in performance to EBR in this configuration.

Figure 11 presents the memory overhead of the Natarajan-Mittal tree under a 50% read and 50% write workload. In both Figure 11a and Figure 11b, we observe consistent trends in memory usage. HP and HPopt maintain the lowest number of nodes in the limbo state, which we attribute to its strict and conservative reclamation guarantees. In contrast, EBR exhibits the highest memory overhead, reflecting its more relaxed and delayed reclamation behavior.

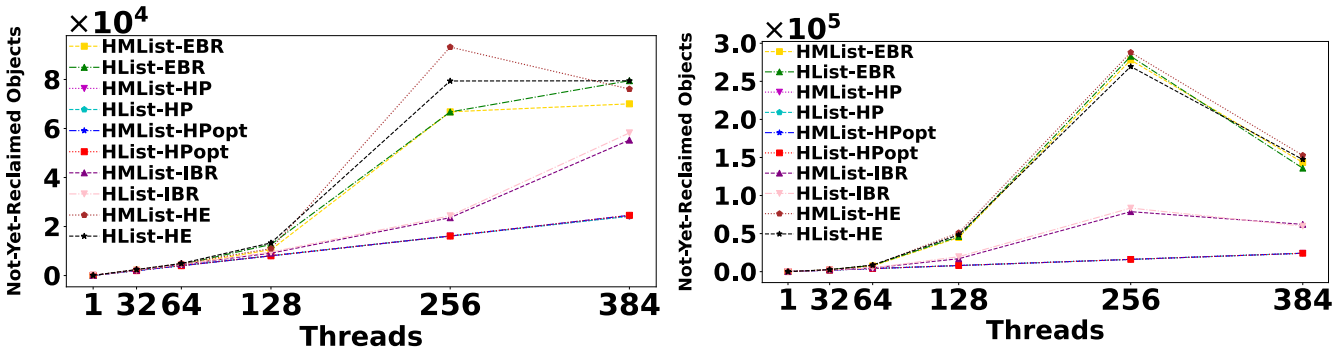
We additionally evaluate the Natarajan-Mittal tree on a large key range of 50 million keys, so that the tree does not fit in the CPU cache. Figure 12a shows that the overall throughput decreases relative to smaller key ranges as expected due to the increase in the traversal depth and reduced cache locality. However, relative trends of SMR schemes remain consistent with the smaller-key experiments. In particular,

Table 2. Restart Statistics for HP, Key Range = 10,000.

	The Harris-Michael List			Harris' List		
Threads	**1**	**64**	**256**	**1**	**64**	**256**
Restarts (per 10 sec)	0	637938	3373950	0	6	59
Ops/sec	74992	1925414	4119613	78290	2025492	4424534
	0%	**3.31%**	**8.19%**	≈ **0%**		

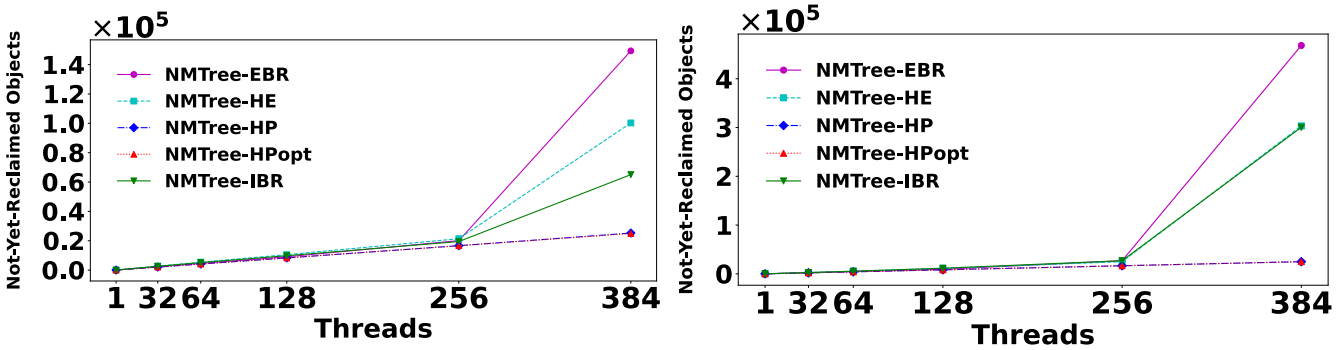
IBR and Hyaline-1S remain highly competitive, often approaching the throughput of EBR even at high thread counts. Figure 12b reports the average number of not-yet-reclaimed objects. At the highest thread count (384), EBR exhibits the largest number of unreclaimed objects. In contrast, HP and HPopt maintain the lowest memory footprint, reflecting their strict and robust reclamation behavior.

Finally, in Table 2, we provide empirical restart behavior for the Harris-Michael list and Harris' list with HP for the scenario in Figure 8b. We observe that for the Harris-Michael list, the restart rate increases from 0% at 1 thread to 8.19% at 256 threads. In contrast, Harris' list exhibits ≈ 0% restart rate across any thread count. Since restarts are expensive and require re-traversing the list from the head, they incur substantial overheads. This, combined with a reduced number of CAS during physical unlinking, explains why Harris' list consistently outperforms the Harris-Michael list in Figure 8.



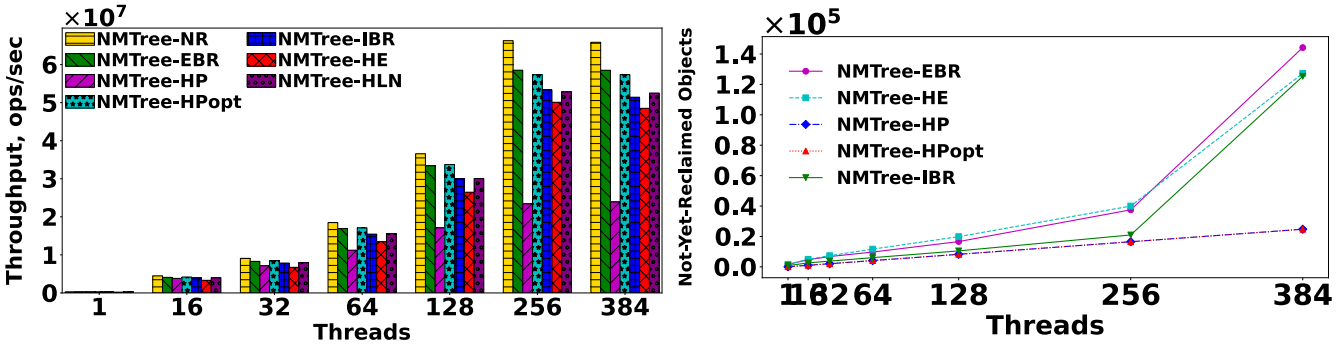
(a) Key Range = 512 **(b)** Key Range = 10,000

Figure 10. Linked List Average Number of Not-Yet-Reclaimed Objects (50% Read - 50% Write): *lower is better.*



(a) Key Range = 128 **(b)** Key Range = 100,000

Figure 11. NMTree Average Number of Not-Yet-Reclaimed Objects (50% Read - 50% Write): *lower is better.*



(a) Throughput (*higher is better*) **(b)** Not-Yet-Reclaimed Objects (*lower is better*)

Figure 12. NMTree (50% Read - 50% Write), Key Range = 50,000,000 (not fitting in L1/L2/L3 cache).

6 Related Work

6.1 Non-Blocking SMR

The ERA theorem [30] demonstrates that at most two out of three properties – (A) **robustness**, (B) **easy integration**, and (C) **wide applicability** – can be achieved simultaneously. Moreover, **strong applicability** currently seems to be available only with SMR schemes that implement some kind of quiescence, similar to EBR. However, quiescence periods inevitably lead to potentially unbounded memory usage.

DEBRA+ [8], QSense [6], ThreadScan [2], ForkScan [1], NBR [31], propose a solution to EBR's robustness (A) problem. However, they all need special OS support, such as scheduler interaction or signal delivery, which requires locks in typical OSs. These schemes satisfy (A) and (C) properties but not (B), which is essential to the strict non-blocking progress. Other schemes that satisfy (A) and (C), e.g., VBR [29], need non-trivial roll-back mechanisms, which are hard to generalize under a uniform library API.

There were also recent attempts to use automatic garbage collection techniques. Generally speaking, garbage collection is subject to the same trade-offs and limitations, discussed above. FreeAccess [9] addresses the problem with (C) and optimistic traversals but forgoes (B) by requiring to divide code into separate read and write phases. OrcGC [10] is another lock-free garbage collector. OrcGC requires mechanisms similar to smart pointers either in the compiler or the language itself; it can also be slower than HP in certain tests.

Other SMR schemes such as interval-based reclamation (IBR) [34], hazard eras (HE) [28], wait-free eras (WFE) [25], Hyaline [26], and Crystalline [27] provide an HP-like interface; they still lack the wide-applicability property (C) but are often faster than HP. SCOT's method to mitigate the problem with (C) is suitable to these SMR schemes as well.

Drop the Anchor (DTA) [7] is a lightweight memory reclamation technique designed specifically for Harris-style singly linked lists. DTA's idea is to reduce the high per-pointer cost of hazard pointers by amortizing protection over multiple node accesses. Instead of protecting every node, DTA periodically records an anchor every few traversed nodes and relies on timestamps, freezing, and a recovery procedure to ensure safety even in the presence of thread failures. By reducing memory barriers and CAS operations, DTA achieves significantly better performance for traversal-heavy workloads. However, its design is closely tied to the linear traversal pattern of linked lists and depends on the ability to freeze and reconstruct contiguous sublists during recovery, making it difficult to adapt to data structures with branching or non-linear access patterns such as trees, skip lists, or hash maps. In contrast, SCOT enables optimistic traversal with the existing robust SMR schemes by modifying the data structure itself to safely validate traversal paths. Thus, while DTA optimizes the HP mechanism for a specific purpose, SCOT expands the applicability of general-purpose SMRs by adapting high-performance non-blocking data structures to remain compatible, preserving both performance and correctness without requiring SMR-specific recovery logic.

Pass-the-buck [16, 17] uses a similar model as HP. Beware & Cleanup (B&C) [13] enhances HP with a limited form of reference counting (RC) for retired records. The key insight is to free a record only when no other records point to it (RC is zero) and no hazard pointer protects it. While potentially addressing optimistic traversals, B&C's retirement algorithm is complex and incurs higher overhead than HP or RC [8].

6.2 Non-Blocking Data Structures

Timothy L. Harris [14] was the first to present a *practical* lock-free linked list. Maurice Herlihy and Nir Shavit showed [18] that read-only optimistic search operations are feasible with Harris' list. Unfortunately, Harris' list is incompatible with HP [3, 9, 10, 19, 26, 27]. Maged M. Michael proposed [20] disabling optimistic traversals by unlinking logically deleted nodes one by one, making it feasible to use the algorithm with HP. SCOT enables support for Harris' original list with HP, HE, IBR, and Hyaline, while using optimistic traversals.

A skip list provides probabilistic logarithmic-time search, insertion, and deletion. Lock-free skip lists [12, 18] resemble Harris and Harris-Michael lists but consist of multiple sublists. However, challenges related to logical deletion and optimistic traversals are identical.

Hash maps store and retrieve key-value pairs in constant time. Lock-free hash map [20] is an array of lists and can be based on either Harris' or the Harris-Michael approach.

Binary search trees are also commonly used for efficient search, insertion, and deletion operations. Ellen at al. tree [11] implements one such approach which is compatible with HP. However, the Natarajan-Mittal tree [24] is significantly faster (around 43% according to [19]) due to its ability to remove multiple concurrently tagged edges in one pass. Prior to our work, as pointed out in [3, 19], the Natarajan-Mittal tree was incompatible with HP, IBR, and other similar schemes.

7 Conclusion

We introduced safe concurrent optimistic traversals (SCOT), a new method which enables support for data structures that are incompatible with HP, IBR, HE, Hyaline-1S, and similar SMRs that lack the wide applicability support. Despite prior beliefs [19] of incompatibility of the Natarajan-Mittal tree and Harris' list with HP, we have demonstrated that not only they are feasible with HP, IBR, HE, and Hyaline-1S, but they can also retain performance benefits when comparing to equivalent data structures without optimistic traversals. We have also presented a mechanism which returns wait-free traversals to SCOT-based data structures, further eliminating the gap with EBR-based implementations.

While SCOT does not apply universally, a wide range of data structures can now be safely implemented with the above-mentioned schemes. EBR can still be a good choice for API simplicity, but SCOT enables an alternative with good performance and compatibility when strong robustness is required. We hope our work inspires further research and reevaluation of remaining challenges in SMR, as limited applicability can be addressed in other ways.

Acknowledgements

A preliminary version of SCOT previously appeared as a brief announcement at SPAA '25 [5].

We would like to thank the anonymous reviewers and our shepherd Trevor Brown for their insightful comments and suggestions, which helped greatly improve this paper.

Data-Availability Statement

The benchmark and data supporting this paper are available on Zenodo [4]. The most up-to-date source code is also available at https://github.com/rusnikola/scot.

References

[1] Dan Alistarh, William Leiserson, Alexander Matveev, and Nir Shavit. 2017. Forkscan: Conservative Memory Reclamation for Modern Operating Systems. In *Proceedings of the 12th European Conference on Computer Systems* (Belgrade, Serbia) *(EuroSys '17)*. 483–498. doi:10.1145/3064176.3064214

[2] Dan Alistarh, William M. Leiserson, Alexander Matveev, and Nir Shavit. 2015. ThreadScan: Automatic and Scalable Memory Reclamation. In *Proceedings of the 27th ACM Symposium on Parallelism in Algorithms and Architectures* (Portland, Oregon, USA) *(SPAA '15)*. 123–132. doi:10.1145/2755573.2755600

[3] Daniel Anderson, Guy E. Blelloch, and Yuanhao Wei. 2021. Concurrent deferred reference counting with constant-time overhead. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI 2021)*. ACM, New York, NY, USA, 526–541. doi:10.1145/3453483.3454060

[4] Md Amit Hasan Arovi and Ruslan Nikolaev. 2025. Artifact for PPoPP'26. https://doi.org/10.5281/zenodo.17707898

[5] Md Amit Hasan Arovi and Ruslan Nikolaev. 2025. Brief Announcement: SCOT: Fix non-blocking data structures, not memory reclamation. In *Proceedings of the 37th ACM Symposium on Parallelism in Algorithms and Architectures* (Portland, OR, USA) *(SPAA '25)*. ACM, New York, NY, USA, 603–607. doi:10.1145/3694906.3743348

[6] Oana Balmau, Rachid Guerraoui, Maurice Herlihy, and Igor Zablotchi. 2016. Fast and Robust Memory Reclamation for Concurrent Data Structures. In *Proceedings of the 28th ACM Symposium on Parallelism in Algorithms and Architectures* (Pacific Grove, California, USA) *(SPAA '16)*. ACM, New York, NY, USA, 349–359. http://doi.acm.org/10.1145/2935764.2935790

[7] Anastasia Braginsky, Alex Kogan, and Erez Petrank. 2013. Drop the anchor: lightweight memory management for non-blocking data structures. In *Proceedings of the 25th Annual ACM Symposium on Parallelism in Algorithms and Architectures* (Montréal, Québec, Canada) *(SPAA '13)*. ACM, New York, NY, USA, 33–42. doi:10.1145/2486159.2486184

[8] Trevor Alexander Brown. 2015. Reclaiming Memory for Lock-Free Data Structures: There Has to Be a Better Way. In *Proceedings of the 2015 ACM Symposium on Principles of Distributed Computing* (Donostia-San Sebastián, Spain) *(PODC '15)*. 261–270. doi:10.1145/2767386.2767436

[9] Nachshon Cohen. 2018. Every data structure deserves lock-free memory reclamation. *Proc. ACM Program. Lang.* 2, OOPSLA, Article 143 (oct 2018), 24 pages. doi:10.1145/3276513

[10] Andreia Correia, Pedro Ramalhete, and Pascal Felber. 2021. OrcGC: Automatic Lock-Free Memory Reclamation. In *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP '21)*. ACM, 205–218.

[11] Faith Ellen, Panagiota Fatourou, Eric Ruppert, and Franck van Breugel. 2010. Non-blocking binary search trees. In *Proceedings of the 29th ACM SIGACT-SIGOPS Symposium on Principles of Distributed Computing* (Zurich, Switzerland) *(PODC '10)*. ACM, New York, NY, USA, 131–140. doi:10.1145/1835698.1835736

[12] Keir Fraser. 2004. *Practical lock-freedom*. Technical Report. Univ. of Cambridge, Computer Laboratory. http://www.cl.cam.ac.uk/techreports/UCAM-CL-TR-579.pdf

[13] Anders Gidenstam, Marina Papatriantafilou, Hakan Sundell, and Philippas Tsigas. 2009. Efficient and Reliable Lock-Free Memory Reclamation Based on Reference Counting. *IEEE Trans. Parallel Distrib. Syst.* 20, 8 (Aug. 2009), 1173–1187. doi:10.1109/TPDS.2008.167

[14] Timothy L. Harris. 2001. A Pragmatic Implementation of Non-blocking Linked-Lists. In *Proceedings of the 15th International Conference on Distributed Computing (DISC '01)*. Springer-Verlag, Berlin, Heidelberg, 300–314.

[15] Thomas E. Hart, Paul E. McKenney, Angela Demke Brown, and Jonathan Walpole. 2007. Performance of memory reclamation for lockless synchronization. *J. Parallel and Distrib. Comput.* 67, 12 (2007), 1270 – 1285. doi:10.1016/j.jpdc.2007.04.010

[16] Maurice Herlihy, Victor Luchangco, Paul Martin, and Mark Moir. 2005. Nonblocking memory management support for dynamic-sized data structures. *ACM Trans. Comput. Syst.* 23, 2 (May 2005), 146–196. doi:10.1145/1062247.1062249

[17] Maurice Herlihy, Victor Luchangco, and Mark Moir. 2002. The Repeat Offender Problem: A Mechanism for Supporting Dynamic-Sized, Lock-Free Data Structures. In *Proceedings of the 16th International Conference on Distributed Computing (DISC '02)*. Springer-Verlag, Berlin, Heidelberg, 339–353.

[18] Maurice Herlihy and Nir Shavit. 2012. *The Art of Multiprocessor Programming, Revised Reprint* (1st ed.). Morgan Kaufmann Publishers Inc., San Francisco, CA, USA.

[19] Jaehwang Jung, Janggun Lee, Jeonghyeon Kim, and Jeehoon Kang. 2023. Applying Hazard Pointers to More Concurrent Data Structures. In *Proceedings of the 35th ACM Symposium on Parallelism in Algorithms and Architectures* (Orlando, FL, USA) *(SPAA '23)*. ACM, New York, NY, USA, 213–226. doi:10.1145/3558481.3591102

[20] Maged M. Michael. 2002. High performance dynamic lock-free hash tables and list-based sets. In *Proceedings of the Fourteenth Annual ACM Symposium on Parallel Algorithms and Architectures* (Winnipeg, Manitoba, Canada) *(SPAA '02)*. ACM, New York, NY, USA, 73–82. doi:10.1145/564870.564881

[21] Maged M. Michael. 2004. Hazard pointers: safe memory reclamation for lock-free objects. *IEEE Transactions on Parallel and Distributed Systems* 15, 6 (June 2004), 491–504.

[22] Microsoft. 2025. Mimalloc allocator. https://github.com/microsoft/mimalloc.

[23] Hassan Mujtaba. 2022. AMD EPYC 9654 Genoa 96 Core CPU Benchmarked In Cache & Memory Benchmark - Over 2x Inter-Cache Bandwidth Versus Milan-X & Up To 30 TB/s L1 Transfer Rates. https://wccftech.com/amd-epyc-9654-96-core-genoa-cpu-tears-apart-milan-x-sapphire-rapids-cache-latency-benchmarks-up-to-30-tbps-over-2x-performance-increase.

[24] Aravind Natarajan and Neeraj Mittal. 2014. Fast concurrent lock-free binary search trees. In *Proceedings of the 19th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming* (Orlando, Florida, USA) *(PPoPP '14)*. ACM, New York, NY, USA, 317–328. doi:10.1145/2555243.2555256

[25] Ruslan Nikolaev and Binoy Ravindran. 2020. Universal Wait-Free Memory Reclamation. In *Proceedings of the 25th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming* (San Diego, California) *(PPoPP '20)*. ACM, New York, NY, USA, 130–143. doi:10.1145/3332466.3374540

[26] Ruslan Nikolaev and Binoy Ravindran. 2021. Snapshot-free, transparent, and robust memory reclamation for lock-free data structures. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI 2021)*. ACM, New York, NY, USA, 987–1002. doi:10.1145/3453483.3454090

[27] Ruslan Nikolaev and Binoy Ravindran. 2024. A Family of Fast and Memory Efficient Lock- and Wait-Free Reclamation. In *Proceedings of the 2024 ACM SIGPLAN International Conference on Programming Language Design and Implementation* (Copenhagen, Denmark) *(PLDI '24)*. ACM, New York, NY, USA, 235:1–235:25. doi:10.1145/3658851

[28] Pedro Ramalhete and Andreia Correia. 2017. Brief Announcement: Hazard Eras - Non-Blocking Memory Reclamation. In *Proceedings of the 29th ACM Symposium on Parallelism in Algorithms and Architectures* (Washington, DC, USA) *(SPAA '17)*. ACM, New York, NY, USA, 367–369. doi:10.1145/3087556.3087588

[29] Gali Sheffi, Maurice Herlihy, and Erez Petrank. 2021. VBR: Version Based Reclamation. In *35th International Symposium on Distributed Computing, DISC 2021, October 4-8, 2021, Freiburg, Germany (Virtual Conference) (LIPIcs, Vol. 209)*. Schloss Dagstuhl - Leibniz-Zentrum für

Informatik, 35:1–35:18. doi:10.4230/LIPIcs.DISC.2021.35

[30] Gali Sheffi and Erez Petrank. 2023. The ERA Theorem for Safe Memory Reclamation. In *Proceedings of the 2023 ACM Symposium on Principles of Distributed Computing* (Orlando, FL, USA) *(PODC '23)*. ACM, New York, NY, USA, 102–112. doi:10.1145/3583668.3594564

[31] Ajay Singh, Trevor Brown, and Ali Mashtizadeh. 2021. NBR: neutralization based reclamation. In *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming* (Virtual Event, Republic of Korea) *(PPoPP '21)*. ACM, New York, NY, USA, 175–190. doi:10.1145/3437801.3441625

[32] Shahar Timnat, Anastasia Braginsky, Alex Kogan, and Erez Petrank. 2012. Wait-Free Linked-Lists. In *Proceedings of the International Conference on Principles of Distributed Systems (OPODIS)*. Springer Berlin Heidelberg, Berlin, Heidelberg, 330–344. doi:10.1007/978-3-642-35476-2_23

[33] Shahar Timnat and Erez Petrank. 2014. A Practical Wait-Free Simulation for Lock-Free Data Structures. In *Proceedings of the 19th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming* (Orlando, Florida, USA) *(PPoPP '14)*. ACM, New York, NY, USA, 357–368. https://doi.org/10.1145/2555243.2555261

[34] Haosen Wen, Joseph Izraelevitz, Wentao Cai, H. Alan Beadle, and Michael L. Scott. 2018. Interval-based memory reclamation. *SIGPLAN Not.* 53, 1 (feb 2018), 1–13. doi:10.1145/3200691.3178488

A Artifact Description

A.1 Abstract

Our artifact includes: (1) a Linux VM image deployable with VirtualBox; (2) source code for the benchmark, along with all tested data structures and reclamation schemes; (3) benchmark scripts for running tests and generating charts. We include detailed instructions for replicating the results in README.txt.

A.2 Artifact check-list (meta-information)

- **Algorithm:** New algorithm, SCOT.
- **Program:** Benchmark with HList, HMList, and NMTree.
- **Compilation:** clang++ 14.0.0 with -O3 optimizations.
- **Binary:** Linux ELF (x86_64) executables.
- **Run-time environment:** Ubuntu 22.04.5 LTS.
- **Execution:** The execution time is passed through a program parameter.
- **Output:** Section 5 plots produced as PDF/SVG files.
- **Experiments:** Scripts run all presented test cases.
- **Workflow frameworks used?:** No.
- **Publicly available?:** Yes.
- **Code licenses:** 2-BSD and 3-BSD.

A.3 Dependencies

A.3.1 Hardware dependencies. Please ensure that your system (VM) is equipped with a sufficient number of CPUs and ample memory. Our 128-core machine (256 hardware threads) has 384GB of physical RAM, and we recommend at least 128GB, especially if running tests for the NR baseline.

A.3.2 Software dependencies. In our setup with Ubuntu, the following packages were installed: git, build-essential, clang, libstdc++-11-dev, libmimalloc-dev, unzip, zip, python3-pip, libcairo2. We also installed the following python3 packages via pip: numpy, matplotlib, cairosvg, pandas.

A.4 Compilation

```
cd ./scot/SCOT
make
```

A.5 Experiment workflow

- Compile the benchmark.
- Run tests. We provide source.sh which runs all tests presented in the paper.
- For individual tests, you can also invoke tests directly. For example, for a lock-free list test with EBR (2 seconds, 4 threads):

  ```
  ./bench listlf 2 512 1 50 25 25 EBR 4
  ```

  (You can see all options by running ./bench.)
- Plot the results. See below.

A.6 Evaluation

HList and NMTree for HP/HPO/HE/IBR/Hyaline are new implementations that use the SCOT technique; they constitute the main contribution of the paper (both LF and WF versions for HList). Conversely, HMList, which is unlike HList was already feasible for all SMR schemes, and EBR/NR schemes (across all data structures) represent existing baselines.

Running all tests:

For full-blown tests (can take up to 10 hours and require a lot of RAM, e.g., at least 128GB is recommended), execute:

```
cd ./scot/Scripts
nohup ./source.sh &
```

For lightweight tests (which run 2 hours and consume less memory), you can skip the NR baseline:

```
cd ./scot/Scripts
nohup ./source_lightweight.sh &
```

Output results will be located in ./scot/Data for the corresponding data structure: listlf (lock-free list), listwf (list with wait-free traversals), tree (Natarajan-Mittal tree). Also, ./scot/run_XXX directory will contain intermediate files. Finally, ./scot/Scripts/run.log is the standard output (log) file.

Drawing PDF plots:

```
cd ./scot/Scripts
python3 generate_charts.py
```

This script scans ./scot/Data/*, processes all list (lock-free and wait-free versions) and tree benchmark outputs, and produces the corresponding SVG and PDF charts in the *_charts subdirectories. By default, this command uses all available thread counts present in the result files, but the 'paper' parameter enables paper-optimized settings.

Received 2025-09-02; accepted 2025-11-10